

School of Science, Computing and Engineering Technologies
Swinburne University of Technology
SWE30003 Software Architectures and Design
Tutorial Week 6 (OO Design)

Please carefully read the description of the well-known problem of the *Towers of Hanoi*:

“In the jungle outside Hanoi there is a community of monks whose duty is to move rings stacked on poles according to certain rules. There are 64 rings, all of different sizes. Originally they were all stacked neatly on top of each other on a single pole: the largest ring at the bottom, and then the remaining rings in decreasing size, with the smallest ring at the top. There are three poles in the jungle and the rings are to be moved from the first pole onto the third pole. Only one ring may be moved at a time. The rings may be placed on any of the three poles, but a ring may not be placed on top of a smaller ring. It is said that when the monks are finished moving the rings so that they are all stacked in the correct order on the third pole, the world will end.”

You are given the task to come up with an *object-oriented design* of an application that takes a number of rings as argument, say n , and produces a list of moves (e.g., “move ring on top of pole 1 to pole 3”) that solves the Towers of Hanoi for n rings. To facilitate the analysis, you can initially assume that you have an *even* number of rings.

In order to do so, form groups of three to four students and address the following questions:

1. Carefully analyze the problem description and identify *candidate classes*.

=> By reading the problem description, we look for nouns that have independent behavior or state:

- Ring: Represents a single disk. It must know its own size.
- Pole (or Peg): A container that holds a collection of rings. It follows a Stack (LIFO) data structure rule (you can only add or remove from the top).
- HanoiSolver (or Monk): The coordinator object that knows the recursive movement strategy/rules and directs the poles.
- MoveLog: A simple value object used to record and display a single valid action (e.g., "Move top ring from Pole 1 to Pole 3").

2. For each of the identified candidate classes, create a CRC (Class Responsibility Collaboration) Card and briefly describe the purpose of each class at the back of the card.

=> **1. Class: HanoiApp (Controller)**

- **Purpose (Back of Card):** Acts as the system orchestrator and execution entry point. It processes the input n (number of rings), instantiates the poles, populates the source pole with the initial ring stack, and coordinates the simulation's movement loop.

Front of Card:

Responsibilities	Collaborators
Initialize the environment based on the specified number of rings (n)	Pole, Ring
Create 3 Pole instances and configure their spatial/directional links	Pole
Instantiate n Ring objects and push them onto the initial source pole	Ring, Pole
Maintain the execution loop until all rings are successfully moved to the destination pole	Ring, Pole

2. Class: Pole

- **Purpose (Back of Card):** Represents a physical pole in the game. It operates as a specialized stack structure that stores a collection of rings, enforces placement rules (preventing a larger ring from being placed on top of a smaller one), and knows its immediate left and right neighbors.

Front of Card:

Responsibilities	Collaborators
Maintain a sequential stack of rings currently resting on the pole	Ring
Accept incoming rings and validate that size constraints are not violated	Ring

Pop and release the top ring when a move is requested	Ring
Maintain references to adjacent Pole objects (establishing a circular layout)	Pole
Expose the size of its topmost ring or report if it is empty	Ring

3. Class: Ring

- **Purpose (Back of Card):** Represents an individual ring characterized by its size. It encapsulates the localized intelligence of the movement rules: each ring tracks its own fixed direction (left or right). The smallest ring (Size 1) acts as a specialized pacing object that drives the application's flow.

Front of Card:

Responsibilities	Collaborators
Maintain its immutable size attribute to ensure valid stacking placement	None
Store its assigned movement direction (Left vs. Right) determined at initialization	Pole
Query its current pole for the neighbor corresponding to its direction and move itself to it	Pole
Format and output its action string (e.g., "move ring from pole 1 to pole 3")	None

3. Given the problem description, identify responsibilities and carefully assign them to the classes identified in Step 1. Discard any unnecessary classes.

Note: you may have to add a new class(es) responsible for setting up the poles, creating the required number of rings, and starting the move of rings. Also, the ring at the top of the initial pole may have different responsibilities than the other rings. Further, a given ring will always move in the *same* direction, that is, either left or right.

1. Environment Setup and Orchestration

- As suggested by the note, a dedicated setup class (HanoiApp) is introduced.
- During initialization, HanoiApp links the three Pole objects together in a circular structure (Pole 1 <-> Pole 2 <-> Pole 3 <-> Pole 1). This allows the moving pieces to navigate dynamically without requiring a central god-class to manually calculate grid coordinates.

2. Specialized Behavior of the Top Ring

- **Context:** The tutorial notes that the ring initially at the top may have unique responsibilities.
- **Design Strategy:** The smallest ring (Size 1) is assigned the role of the system "pacer." In a Towers of Hanoi sequence, the smallest ring moves on every single alternate turn. HanoiApp leverages this by alternating the loop turns between:
 1. Commanding **Ring 1** to execute its predefined directional move.
 2. Executing the single remaining valid legal move possible between the other two poles.

3. Encapsulating Fixed Directional Movement

- **Context:** A given ring must always move in the exact same direction (left or right).
- **Design Strategy:** When HanoiApp generates the rings, it computes and embeds a fixed direction state variable directly into each Ring object:
 - If the total number of rings (\$n\$) is **even**, the smallest ring is programmed to always move **Right**.
 - If \$n\$ is **odd**, the smallest ring is programmed to always move **Left**.
 - When a ring is commanded to move, it simply asks its current Pole for the appropriate neighbor matching its internal direction state, eliminating complex nested decision trees during runtime.

4. Given your current “solution”, perform a walk-through where each member of your small group plays the role of one (or possibly more) objects. What do you learn from this walk through?

=> To verify our responsibility assignments, our group conducted a role-play simulation with a configuration of ($n = 4$) rings (an even number):

- Hieu played the role of HanoiApp (The Orchestrator).
- Dang played the role of all three Pole instances (connected in a circular structure).
- Hoang played the role of the Ring objects (specifically Ring 1, acting as the Pacer).

What we learn from the walk-through:

1. Poles are passive containers: A Pole cannot move a ring by itself because it doesn't know the overall game strategy or where the other poles are. It can only say "Yes, you can take my top ring" or "No, you cannot put that big ring on my small ring."
2. Rings are completely dumb data objects: A Ring does not know where it is or where it needs to go. It only holds its size.
3. Centralized Control Problem: In a naive design, the HanoiSolver has to do all the thinking, checking, and moving, making the other objects completely brainless data holders.

5. Based on the observations from Step 4, reconsider your “solution”: do you need different classes? Do you have *evenly distributed system intelligence*? Do you need to re-assign responsibilities? Can you reduce the amount of communication between various objects? You may need to iterate steps 1 to 4 a few times to get your solution “right”.

=>

We do not need different classes because our primary classes (HanoiApp, Pole, Ring) are structurally sufficient. However, we realized we did not need an independent, heavy "Solver" class. Instead, the entry point application class (HanoiApp) can easily run the execution loop itself by coordinating the active objects, keeping our class hierarchy clean.

Evenly distributed system intelligence:

- Before: In a traditional centralized design, the controller class holds 100% of the logic (knowing all the rules, checking sizes, and moving data arrays), making Ring and Pole completely brainless data holders.
- After Reconsideration: Intelligence is now beautifully distributed using Responsibility-Driven Design:
 - Ring 1 (Pacer): Holds the active runtime responsibility to drive the alternate-turn pace of the system.
 - Pole: Holds the defensive validation intelligence. It is responsible for protecting its own data stack boundaries by checking if an incoming ring size is legal via its `canAccept()` behavior.

We need to re-assign responsibilities. We shifted the movement mechanics directly onto the Ring objects. Instead of the application controller manually lifting a ring and

calculating its grid coordinates, a Ring is now responsible for knowing its own fixed direction state variable (Left vs. Right) and executing its own hop when commanded.

We reduce the amount of communication between various objects drastically. By configuring the three Pole instances in a circular linked structure (where each pole maintains a reference to its immediate left and right neighbors), we eliminated a massive amount of back-and-forth messaging.

Instead of HanoiApp querying Pole 1, checking the top ring, calculating the destination index, and calling Pole 2, the communication path is now a clean chain reaction:

HanoiApp —> Ring —> Current Pole —> Target Neighbor Pole

This optimization eliminates nested runtime if/else decision trees and significantly minimizes object coupling.

6. Any questions from weekly submissions? => Perhaps not